Recap

Speed test tools vary in terms of workload and targets

Better accuracy when the vantage point and the target belong to the same provider network

Web applications can be tested using the Speedometer tool

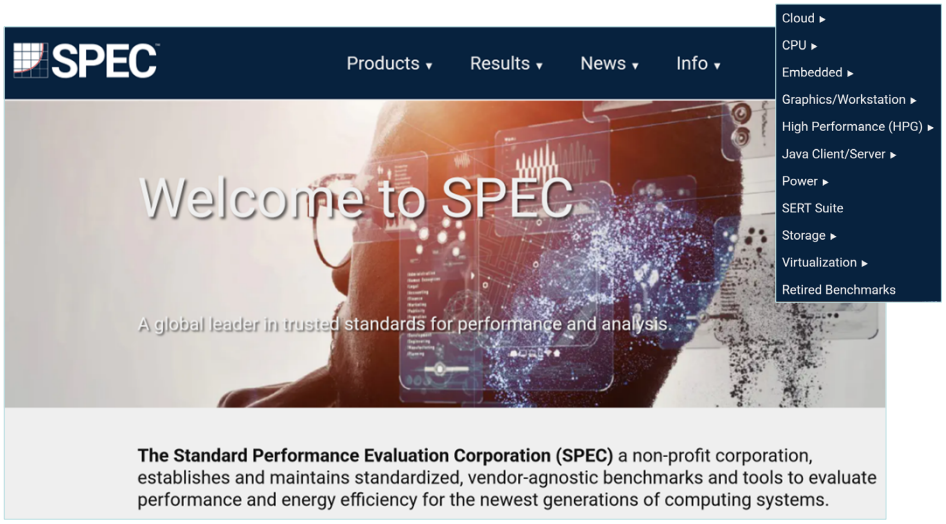
Benchmark frameworks

Non-profit organizations promote standard benchmarks to be used as a reference point for comparing hardware and software solutions of different providers/vendors

Two main objectives:

1. Creating “good” benchmarks, i.e., benchmarks representative of different application domains

2. Creating a process for reviewing and monitoring the benchmarks, i.e., strict rules associated with the runs of these benchmarks to ensure the reproducibility and repeatability of the performance results

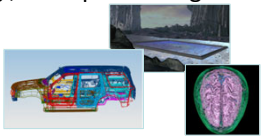


The benchmark measures the 3D graphics performance of systems running under the OpenGL, DirectX, and Vulkan APIs

Frequently used as the basis for performance measurement and estimation of new hardware, e.g., GPUs

Free for non commercial usage (latest release Dec. 2025)

The benchmark workloads represent graphics content and behavior from workstation-class real-world industrial use cases, e.g., complex design scenarios, animations, visualizations



Subtest scores are measured in Frames per Second

Workload composite scores are calculated by taking a weighted geometric mean of the constituent subtest scores

Details of some SPECviewperf 15 workloads

3dsmax-08 v0.7
The 3dsmax-08 workload, derived from Autodesk 3ds Max 2023 using DirectX 11, features diverse models from the SPECapc benchmark and other sources. It showcases rendering styles such as realistic, shaded, wireframe, and lesser-used interest rendering modes like facets, graphite, and clay. Multiple viewsets and animations, including model spins and camera fly-throughs, simulate real-world design workflows. This workload evaluates graphics performance for architectural, engineering, and creative visualizations.

blender-01 v0.5
The blender-01 workload, derived from Blender 3.6 LTS using OpenGL, tests performance across varied rendering techniques using metroverse and classroom scenes. Subtests include wireframe, shaded, x-ray modes, and material rendering with the Eevee engine. It explores quad viewports and advanced rendering effects like shadows and randomized object colors. These tests reflect real-world scenarios for animation, modeling, and visualization workflows.

catia-07 v0.8
The catia-07 workload, derived from Dassault Systèmes' CATIA V5 and 3DEXPERIENCE CATIA applications using OpenGL, features models ranging from 5.1 to 21 million vertices. It includes rendering modes such as anti-aliasing, shaded, and shaded with edges. Subtests simulate complex design scenarios, including cars, jet engines, and loft jets in multi-viewport and varied lighting configurations, offering a comprehensive test for engineering workflows.

creo-04 v0.5
The creo-04 workload, created from PTC's Creo 9™ using OpenGL, tests performance across diverse rendering scenarios. It features detailed models like Scorpion, Submarine, and World Car rendered in modes such as shaded, wireframe on shaded, and hidden line. Advanced effects like reflections, ambient occlusion, bump mapping, and transparency are applied, with anti-aliasing settings ranging from none to 8x MSAA. The workload evaluates real-world engineering and design workflows, with models ranging from 20 to 48 million vertices.

53

Minimum requirements

Component	Requirement
OS	*Windows 11 Pro version 23H2*
CPU	*Intel 12th Gen* or *AMD Ryzen 7000 series* or newer
Graphics	*AMD*, *Intel*, or *NVIDIA* GPU with *4GB or greater shared or dedicated memory*, *OpenGL 4.5*, *DirectX 12*, *Vulkan 1.1* and *Hardware H.265* encoding 🔗 See Workload Requirements
RAM	*15GB available system RAM* and at least *1GB/thread*
Disk Space	At least *100GB free space* for install and execution
Display Resolution	Minimum 1920 x 1080

Workload	Requirement(s)
catia-07	*at least 6GB video memory*
energy-04	*at least 4GB video memory*
enscape-01	*Hardware Ray Tracing Support; at least 6GB video memory*
unreal_engine-01	*An AMD, Intel, or NVIDIA graphics card that supports DirectX 12 (with Shader Model 6.6 atomics), hardware ray tracing (DXR Tier 1.1 and Resource Binding Tier 3); at least 8 GB of video memory.*

54

Some results

LENOVO P3 Tower Gen2	Intel(R) Core(TM) Ultra 9 285K	1x Intel(R) Graphics 1x NVIDIA RTX PRO 6000 Blackwell Workstation Edition	Nov 11 2025	15.0.1-rc2

System Configuration	
Manufacturer	LENOVO
Model	P3 Tower Gen2
CPU	Intel(R) Core(TM) Ultra 9 285K
Memory	128.00 GB @ 4400 MHz
GPU	1x Intel(R) Graphics 1x NVIDIA RTX PRO 6000 Blackwell Workstation Edition
Display	P27 27.2" (3840x2160)
DisplayDPI	96 (Scale Factor: 100%)
Storage	SKHynix_HFS001TFM9X179N 953.86 GB - SCSI
OS	Microsoft Windows 11 Pro (26100)

55

Scores

Workload Scores	
Workload	**Composite Score**
3dsmax-08	108.20
blender-01	111.00
catia-07	119.07
creo-04	246.09
energy-04	159.64
enscape-01	73.15
maya-07	169.59
medical-04	172.64
snx-05	109.30
solidworks-08	208.69
unreal_engine-01	90.69

56

More details

Medical-04

The medical-04 workload evaluates medical visualization techniques using the Tuvok library and OpenGL API. It features slice rendering and raycasting with 1D and 2D transfer functions, exploring datasets from a beating heart, brain, and alligator. Clipping planes and varying voxel densities test performance in medical imaging workflows, simulating scenarios like MRI and CT analysis.

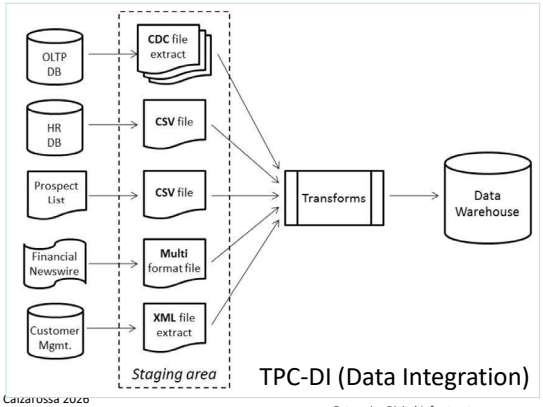
Composite Score: 172.64

Graphics Renderer: RTX PRO 6000 Blackwell Workstation Edition

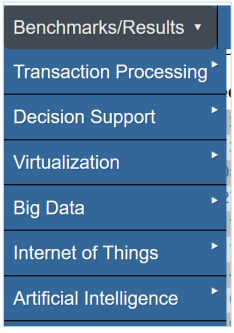
Index	Description	Weight	Result (FPS)
1	 Slice beating heart with 1D transfer	10.00	285.79
2	 Raycasting of a beating heart with 1D transfer	10.00	931.70
3	 Slice beetle with 1D transfer	10.00	97.90
4	 Raycasting of a beetle with 1D transfer	10.00	120.35
5	 Raycasting of a brain with 2D transfer	10.00	224.00

TPC®

Transaction Processing Performance Council focusing on data-centric benchmark standards and disseminating objective, verifiable data to industry



TPC-DI (Data Integration)

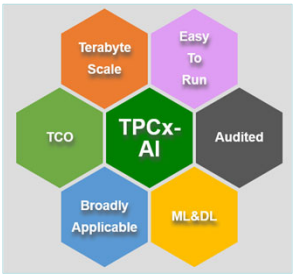


TPC provides benchmark specifications and some tool kits

TPCx-AI

TPCx-AI is a standard benchmark that measures the performance of a machine learning or data science platform

It emulates the behavior of representative industry AI solutions relevant in current production data centers and cloud environments



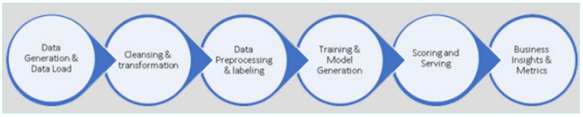
59

TPCx-AI

The benchmark is framework and syntax agnostic and can be implemented in many ways

In the TPCx-AI kit, almost all use cases implemented include data generation, data management, training, scoring and serving phases

Use cases include customer segmentation, sales forecasting, spam detection, price prediction, classification and fraud detection



The execution of the benchmarks follows a well-defined workflow

60

Passive monitoring

Passive monitoring techniques and tools aim at measuring the performance and assessing the efficiency of services/infrastructure while they are operating

Passive monitoring is based on the direct observation of the behavior/operations of a real infrastructure under its actual workload/traffic (when)

Observability refers to understanding the behavior and the performance of a system through observation (why)

Many diverse techniques/tools for passive monitoring

The choice depends on the objectives of the monitoring project, but also on the target and on the usage of the measurements (i.e., online, offline)

61

Logging facilities

Logging facilities collect measurements of all activities and events (including errors and malfunctioning) generated by applications and services

These facilities are typically integrated into operating systems and applications/services

Data being collected must comply with the GDPR

Measurements are organized into records stored into log or trace files (e.g., access, audit) according to different formats

Offline usage of the measurements

Possible usages: analytics, accounting, auditing, marketing, requested by law

Every activity performed over the Internet is tracked and corresponding data is stored somewhere (even in multiple places)

62

Logging facilities: Web logs

Apache access log (stored in a standard format)

```
91.196.152.125 - - [15/Mar/2026:01:54:50 +0100] "GET / HTTP/1.1" 200 6995 "-" "Mozilla/5.0 (X11; Ubuntu; Linux x86_
64; rv:134.0) Gecko/20100101 Firefox/134.0"
52.167.144.146 - - [15/Mar/2026:01:55:15 +0100] "GET /publications/PDF/Proxy.pdf HTTP/1.1" 200 180143 "-" "Mozilla/
5.0 AppleWebKit/537.36 (KHTML, like Gecko; compatible; bingbot/2.0; +http://www.bing.com/bingbot.htm) Chrome/116.0.
1938.76 Safari/537.36"
95.214.55.63 - - [15/Mar/2026:01:56:03 +0100] "GET / HTTP/1.1" 200 3666 "-" "-"
149.86.227.60 - - [15/Mar/2026:02:22:04 +0100] "GET / HTTP/1.1" 200 3666 "-" "Mozilla/5.0"
138.68.130.42 - - [15/Mar/2026:02:27:59 +0100] "SSTP_DUPLEX_POST /sra_{BA195980-CD49-458b-9E23-C84EE0ADCD75}/ HTTP/
1.1" 400 3693 "-" "-"
45.205.1.8 - - [15/Mar/2026:02:33:31 +0100] "GET / HTTP/1.1" 200 6979 "-" "Mozilla/5.0"
176.65.149.235 - - [15/Mar/2026:02:35:41 +0100] "GET / HTTP/1.1" 200 3666 "-" "-"
43.153.47.201 - - [15/Mar/2026:02:47:19 +0100] "GET / HTTP/1.0" 400 535 "-" "-"
138.91.30.48 - - [15/Mar/2026:02:49:04 +0100] "GET /MCC/publications/workloads-clouds-preprint.pdf HTTP/1.1" 200 47
5390 "-" "Mozilla/5.0 AppleWebKit/537.36 (KHTML, like Gecko); compatible; ChatGPT-User/1.0; +https://openai.com/bot
"
138.91.30.61 - - [15/Mar/2026:02:49:16 +0100] "GET / HTTP/1.1" 200 4884 "-" "Mozilla/5.0 AppleWebKit/537.36 (KHTML,
 like Gecko); compatible; ChatGPT-User/1.0; +https://openai.com/bot"
104.210.140.128 - - [15/Mar/2026:02:52:09 +0100] "GET /robots.txt HTTP/1.1" 404 3661 "-" "Mozilla/5.0 (Macintosh; I
ntel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/131.0.0.0 Safari/537.36; compatible; OAI-Searc
hBot/1.0; +https://openai.com/searchbot"
```

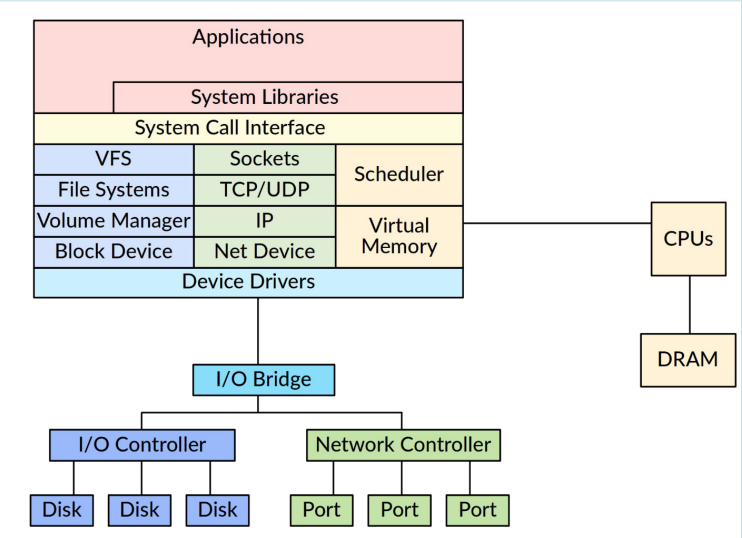
Apache error log

```
[Mon Mar 16 09:29:59.351405 2026] [autoindex:error] [pid 2337:tid 128241146787520] (13)Permis
sion denied: [client 165.232.190.108:50393] AH01275: Can't open directory for index: /var/www
/html/PEG/images/, referer: binance.com
[Mon Mar 16 18:14:55.569307 2026] [core:error] [pid 2338:tid 128242522502848] [client 47.250.
153.97:35214] AH10244: invalid URI path (/cgi-bin/.%2e/.%2e/.%2e/.%2e/.%2e/.%2e/.%2e/.%2
e/.%2e/bin/sh)
[Mon Mar 16 18:14:56.042266 2026] [core:error] [pid 2338:tid 128242402981568] [client 47.250.
153.97:35216] AH10244: invalid URI path (/cgi-bin/%%32%65%%32%65/%%32%65%%32%65/%%32%65%%32%6
5/%%32%65%%32%65/%%32%65%%32%65/%%32%65%%32%65/%%32%65%%32%65/bin/sh)
[Mon Mar 16 18:49:05.369358 2026] [core:error] [pid 2337:tid 128242136635072] [client 178.136
.46.2:50448] AH10244: invalid URI path (/cgi-bin/.%2e/.%2e/.%2e/.%2e/.%2e/.%2e/.%2e/.%2e
/.%2e/bin/sh)
```

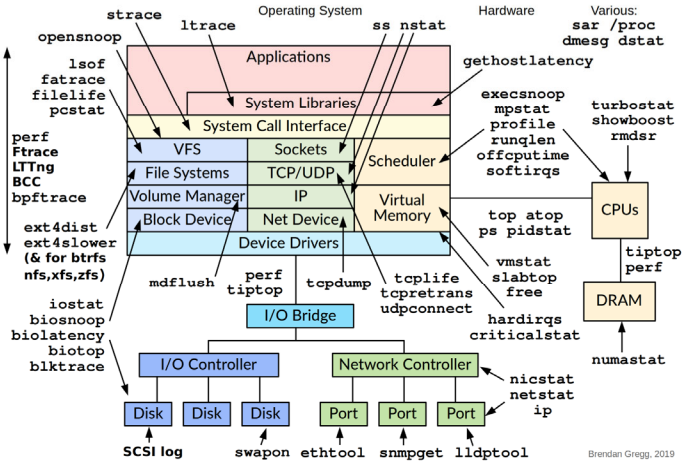

Linux authorization log

```
2026-03-15T00:01:57.388172+01:00 piccione sshd[3447825]: Failed password for root from 2.57.1
22.190 port 60902 ssh2
2026-03-15T00:02:07.440739+01:00 piccione sshd[3447825]: message repeated 3 times: [ Failed p
assword for root from 2.57.122.190 port 60902 ssh2]
2026-03-15T00:02:10.106193+01:00 piccione sshd[3448108]: Invalid user debian from 80.94.95.11
6 port 18846
2026-03-15T00:02:10.208761+01:00 piccione sshd[3448108]: pam_unix(sshd:auth): check pass; use
r unknown
2026-03-15T00:02:10.208988+01:00 piccione sshd[3448108]: pam_unix(sshd:auth): authentication
failure; logname= uid=0 euid=0 tty=ssh ruser= rhost=80.94.95.116
2026-03-15T00:02:11.241472+01:00 piccione sshd[3447825]: Failed password for root from 2.57.1
22.190 port 60902 ssh2
2026-03-15T00:02:12.240617+01:00 piccione sshd[3448108]: Failed password for invalid user deb
ian from 80.94.95.116 port 18846 ssh2
```

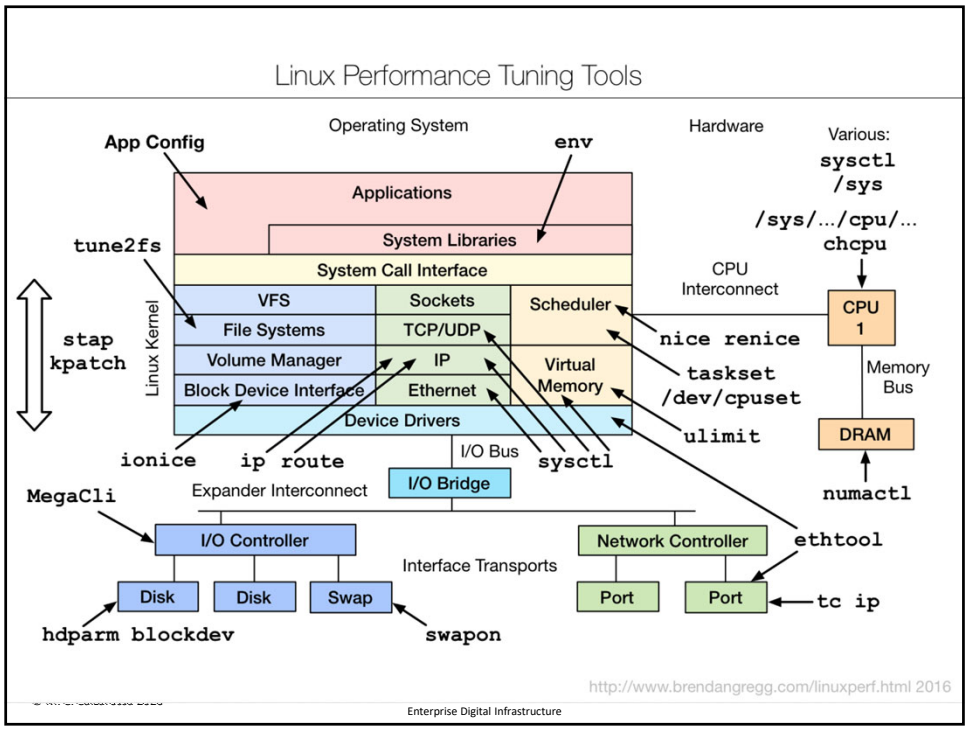
How to measure system components



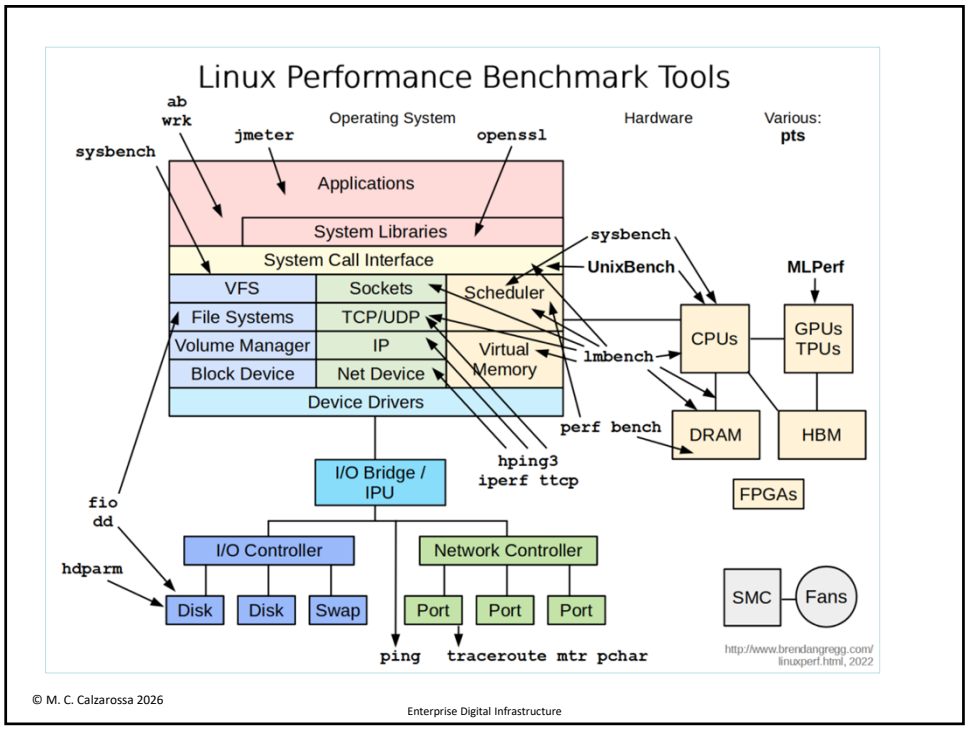
Linux Observability Tools



Brendam Gregg "Systems Performance: Enterprise and the Cloud, 2nd Edition" (2020)


Linux Performance Tuning Tools
Operating System
Hardware
Various:
App Config
env
sysctl
/sys
/sys/.../cpu/...
chcpu
Applications
System Libraries
System Call Interface
tune2fs
CPU
Interconnect
VFS
Sockets
Scheduler
CPU
1
File Systems
TCP/UDP
nice renice
stap
kpatch
Linux Kernel
Volume Manager
IP
taskset
/dev/cpuset
Memory
Bus
Block Device Interface
Ethernet
Virtual
Memory
Device Drivers
ulimit
DRAM
I/O Bus
ionice
ip route
sysctl
I/O Bridge
numactl
MegaCli
Expander Interconnect
I/O Controller
Network Controller
ethtool
Interface Transports
Disk
Disk
Swap
Port
Port
tc ip
hdparm blockdev
swapon
http://www.brendangregg.com/linuxperf.html 2016
Enterprise Digital Infrastructure

67


Linux Performance Benchmark Tools
ab
wrk
Operating System
Hardware
Various:
jmeter
openssl
pts
sysbench
Applications
System Libraries
sysbench
System Call Interface
UnixBench
MLPerf
VFS
Sockets
Scheduler
File Systems
TCP/UDP
CPUs
GPUs
TPUs
Volume Manager
IP
Virtual
Memory
lmbench
Block Device
Net Device
Device Drivers
perf bench
DRAM
HBM
hping3
iperf ttcp
I/O Bridge /
IPU
FPGAs
fio
dd
I/O Controller
Network Controller
hdparm
Disk
Disk
Swap
Port
Port
Port
SMC
Fans
ping
traceroute mtr pchar
http://www.brendangregg.com/
linuxperf.html, 2022
© M. C. Calzarossa 2026
Enterprise Digital Infrastructure

68

Performance monitors

Performance monitors provide measurements about the status and activities of technological infrastructures and services

They are based on the *performance counters* , i.e., data items associated with performance objects, made available by Performance Monitoring Units or by operating systems

They monitor and count *hardware events* and *software events* (e.g., processor clock cycles, cache references, retired instructions, branch misses, page faults, context switches)

Two approaches are applied to collect samples:

- Periodic event polling at a specified time interval
- Continuous event monitoring such that upon occurrence of an event overflows of performance counters trigger interrupts whose handlers enable collecting samples

Performance monitors (cont'd)

The *Performance Counters for Linux* is a kernel-based subsystem that provides a framework for collecting and analyzing performance data [*perf* command]

The *Microsoft Windows Performance Counters* provide a high-level abstraction layer used for collecting various kinds of system data [*Performance Monitor* command]

MacOS Instrument is another performance analysis tool

The data varies depending on the performance monitoring hardware and the software configuration of the system

System administrators use performance counters to monitor systems and identify performance issues

Software developers use performance counters to examine the behavior and resources used by their applications

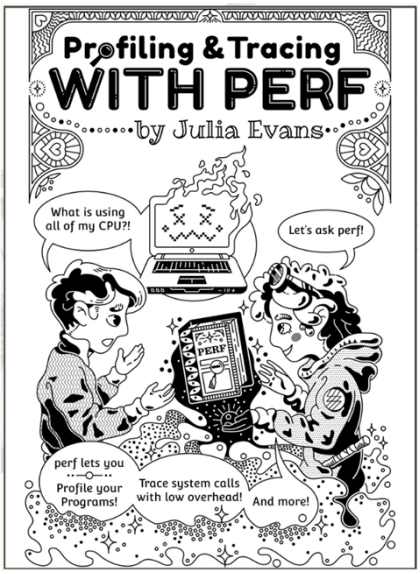
Examples of Linux counters

```
branch-instructions OR branches                    [Hardware event]
branch-misses                                      [Hardware event]
bus-cycles                                         [Hardware event]
cache-misses                                       [Hard
cache-references                                   [Hard
cpu-cycles OR cycles                               [Hard
instructions                                       [Hard
ref-cycles                                         [Hard
stalled-cycles-backend OR idle-cycles-backend      [Hard
stalled-cycles-frontend OR idle-cycles-frontend    [Hard

alignment-faults                                   [Soft
bpf-output                                         [Soft
cgroup-switches                                    [Soft
context-switches OR cs                             [Soft
cpu-clock                                          [Soft
cpu-migrations OR migrations                       [Soft
dummy                                              [Soft
emulation-faults                                   [Soft
major-faults                                       [Soft
minor-faults                                       [Software event]
page-faults OR faults                              [Software event]
task-clock                                         [Software event]

duration_time                                      [Tool event]
user_time                                          [Tool event]
system_time
```

```
L1-dcache-load-misses                              [Hardware cache event]
L1-dcache-loads                                    [Hardware cache event]
L1-dcache-prefetch-misses                          [Hardware cache event]
L1-dcache-store-misses                             [Hardware cache event]
L1-dcache-stores                                   [Hardware cache event]
L1-icache-load-misses                              [Hardware cache event]
LLC-loads                                          [Hardware cache event]
LLC-prefetches                                     [Hardware cache event]
LLC-stores                                         [Hardware cache event]
branch-load-misses                                 [Hardware cache event]
branch-loads                                       [Hardware cache event]
dTLB-load-misses                                   [Hardware cache event]
dTLB-loads                                         [Hardware cache event]
dTLB-store-misses                                  [Hardware cache event]
dTLB-stores                                        [Hardware cache event]
iTLB-load-misses                                   [Hardware cache event]
iTLB-loads                                         [Hardware cache event]
```

```
cache:
  l1d.replacement
       [Counts the number of cache lines replaced in L1 data cache]
  l1d_pend_miss.fb_full
       [Number of cycles a demand request has waited due to L1D Fill Buffer (FB) unavailability]
  l1d_pend_miss.fb_full_periods
       [Number of phases a demand request has waited due to L1D Fill Buffer (FB) unavailability]
  l1d_pend_miss.l2_stall
       [Number of cycles a demand request has waited due to L1D due to lack of L2 resources]
  l1d_pend_miss.pending
       [Number of L1D misses that are outstanding]
  l1d_pend_miss.pending_cycles
       [Cycles with L1D load Misses outstanding]
```


71

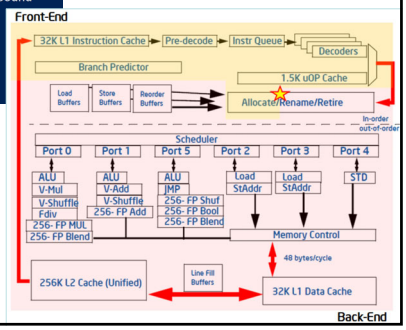


72

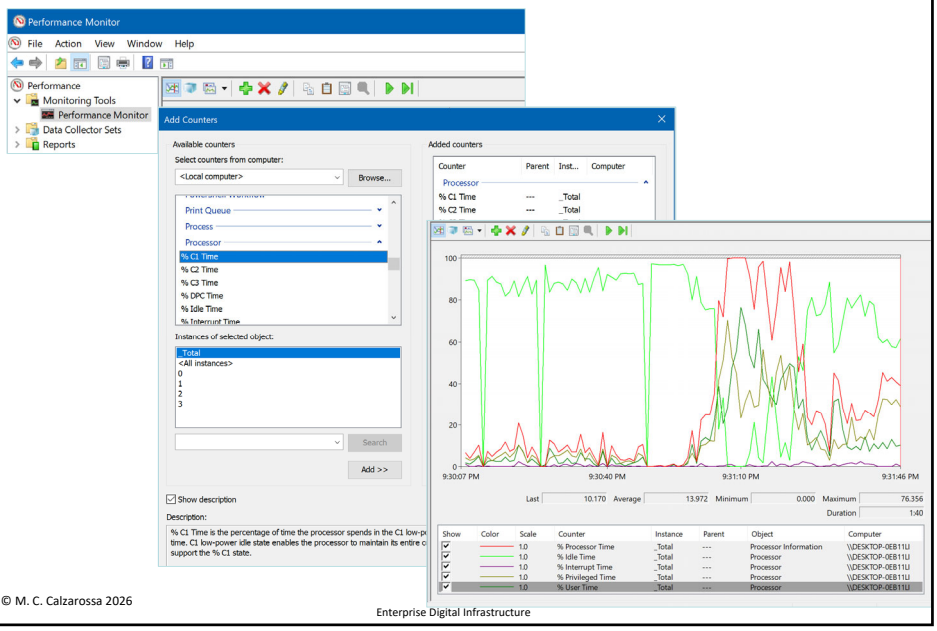
Linux *perf* command

```
Performance counter stats for 'traceroute www.ucsd.edu':

          15.92 msec task-clock                #    0.001 CPUs utilized
             62      context-switches          #    3.894 K/sec
              0      cpu-migrations            #    0.000 /sec
            119      page-faults               #    7.474 K/sec
     12,655,056      cycles                    #    0.795 GHz
      9,402,223      instructions              #    0.74  insn per cycle
      1,763,456      branches                  #  110.757 M/sec
        101,686      branch-misses             #    5.77% of all branches
                     TopdownL1                 #     11.6 %  tma_backend_bound
                                               #     27.3 %  tma_bad_speculation
                                               #     49.3 %  tma_frontend_bound
                                               #     11.8 %  tma_retiring

    15.326041228 seconds time elapsed

     0.004083000 seconds user
     0.014467000 seconds sys
```

Top-down Microarchitecture Analysis - Level 1 [Intel Ice Lake]



Microsoft Windows Performance Monitor



Linux server status: *sysstat* utilities

The `sysstat` utilities are performance monitoring tools for Linux systems

They monitor system performance and usage

These tools are very useful for checking the status of a systems and quickly identifying performance problems (i.e., “crisis” tools)

They collect information about the overall system and of the individual components (e.g., CPU, memory, I/O devices)

“60 seconds tools”: **`uptime`, `vmstat`, `mpstat`, `pidstat`, `iostat`, `top`, …**

Linux server status: examples

```
System load:    0.64              Temperature:             47.0 C
Usage of /home: 30.9% of 2.19TB   Processes:               1087
Memory usage:   20%               Users logged in:         0
Swap usage:     0%                IPv4 address for eno8303: 193.204.34.188
```

```
top - 21:39:30 up 1 day, 10:45,  2 users,  load average: 0.52, 0.42, 0.35
Tasks: 1084 total,   2 running, 1082 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.1 us,  0.2 sy,  0.0 ni, 99.7 id,  0.0 wa,  0.0 hi,  0.0 si,  0.0 st
MiB Mem : 257363.0 total, 199518.3 free,  54818.8 used,   6116.5 buff/cache
MiB Swap:   8192.0 total,   8192.0 free,      0.0 used. 202544.3 avail Mem
```

Performance profilers

Performance profilers are part of the software development ecosystem

They measure the activities and performance of applications/services for identifying (and improving) poorly performing sections of code (*bottlenecks*)

They collect measurements at run time, such as how long the execution of a module/function took, how often it was called, where it was called from, …

Profilers are often based on code *instrumentation*, i.e., monitoring hooks or probes inserted into the source or binary code of the target application/service

They are characterized by two conflicting requirements: *accuracy* vs *intrusiveness*

Specific profilers for the different programming languages (e.g., gprof, Jprof, JVM monitor) or services (e.g., Mozilla Firefox)

Hybrid solutions exploiting probes and performance counters are often applied

77

Python Profilers

Python is equipped with performance profilers that provide useful information to understand where the code spends its time

`cProfile` is a *deterministic* profiler characterized by a "reasonable" overhead that makes it suitable for profiling long-running programs

Deterministic profiling reflects the fact that all events associated with *function calls*, *function returns*, and *exceptions* are monitored and precise timings are made for the intervals between these events

It is not necessary to manually instrument the code since a hook is automatically associated with each event

The profiler provides a detailed breakdown of the number of calls and the total time spent in each function (with a millisecond resolution)

Textual and graphical outputs

78

`cProfile` output

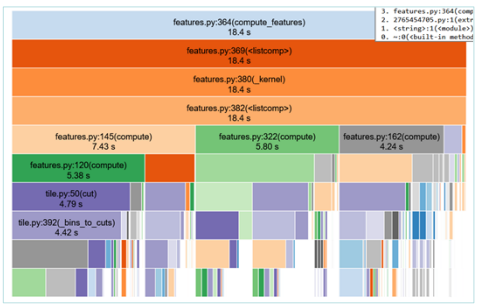
```
         30544211 function calls (30200666 primitive calls) in 18.558 seconds

   Ordered by: cumulative time
   List reduced from 1109 to 23 due to restriction <'features'>

   ncalls  tottime  percall  cumtime  percall filename:lineno(function)
        1    0.000    0.000   18.392   18.392 features.py:364(compute_features)
        1    0.002    0.002   18.388   18.388 features.py:369(<listcomp>)
     2304    0.007    0.000   18.385    0.008 features.py:380(_kernel)
     2304    0.016    0.000   18.369    0.008 features.py:382(<listcomp>)
     2304    0.022    0.000    7.432    0.003 features.py:145(compute)
     2304    0.070    0.000    5.800    0.003 features.py:322(compute)
     2304    0.019    0.000    5.384    0.002 features.py:120(compute)
     2304    0.132    0.000    4.798    0.002 features.py:32(hl_envelopes_idx)
     2304    0.032    0.000    4.238    0.002 features.py:162(compute)
     2304    0.542    0.000    2.295    0.001 features.py:56(<listcomp>)
     2304    0.538    0.000    2.263    0.001 features.py:58(<listcomp>)
     2304    0.094    0.000    0.723    0.000 features.py:296(compute)
     2304    0.004    0.000    0.160    0.000 features.py:107(compute)
     2304    0.038    0.000    0.088    0.000 features.py:20(min_max_normalize)
     2304    0.004    0.000    0.004    0.000 features.py:150(<listcomp>)
     2305    0.001    0.000    0.001    0.000 features.py:366(<genexpr>)
        1    0.000    0.000    0.000    0.000 features.py:355(get_feature_names)
        1    0.000    0.000    0.000    0.000 features.py:357(<listcomp>)
        1    0.000    0.000    0.000    0.000 features.py:293(feature_names)
        1    0.000    0.000    0.000    0.000 features.py:142(feature_names)
        1    0.000    0.000    0.000    0.000 features.py:159(feature_names)
        1    0.000    0.000    0.000    0.000 features.py:104(feature_names)
        1    0.000    0.000    0.000    0.000 features.py:319(feature_names)
```


SnakeViz

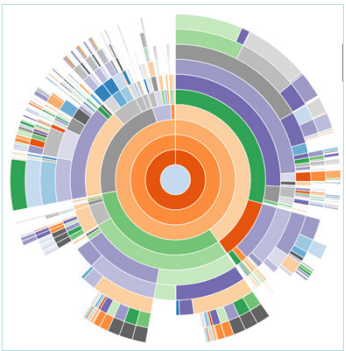
SnakeViz is a viewer for Python profiling data that runs as a web application

Two types of diagrams used to represent hierarchical data: *icicle* and *sunburst*

The fraction of time spent in a function is represented by the extent of a visualization element, i.e., width of a rectangle or angular extent of an arc

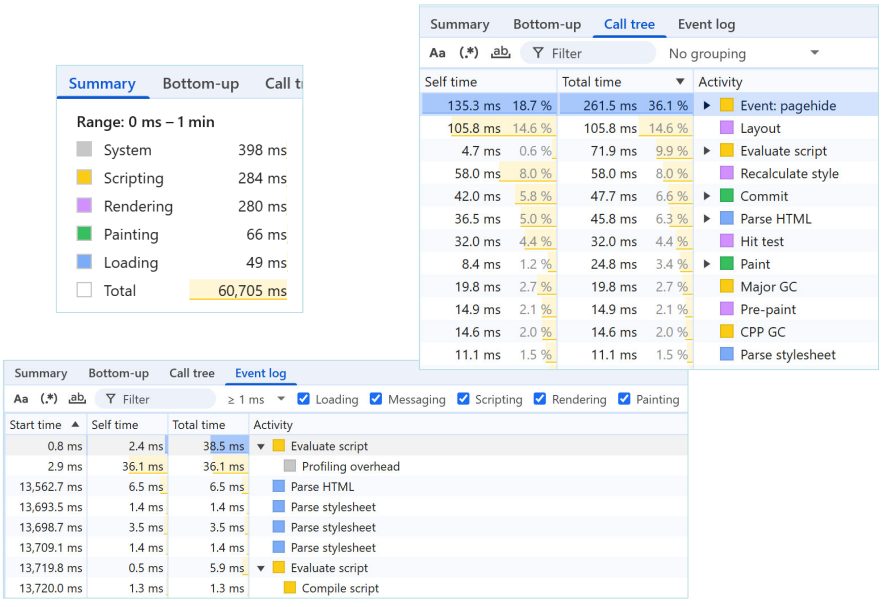


Icicle plot



Sunburst plot

Google Chrome profiler



81

Service/application monitoring

Performance debugging and diagnosis refer to the identification of the inefficient components of the applications (i.e., *bottlenecks*) and the development of appropriate optimization strategies

Two possible approaches to monitor the performance of an application:

- *Performance monitoring*: measurements being collected are associated with *performance counters* and provide a *low-level* view of the behavior and performance characteristics of the application
- *Performance profiling*: measurements being collected are associated with the *individual sections* of the code and provide a detailed *high-level* view of the behavior and performance characteristics of the application

Measurements collected at run time are analyzed offline

82